

Prompt:

8/17/2026, 10:00:00 PM

lanjut Tahap O

Response:

8/17/2026, 10:00:01 PM

Remembering

Clarified Tahap N/O design, dynamic QUBO, and position sizing memory

Worked for 4s

Tahap O - Position Management & Dynamic Rebalancing

Tahap O adalah kelanjutan yang sangat penting dari Tahap N.

Pada Tahap N kita telah menyelesaikan pertanyaan:

"Trade mana yang sebaiknya dibuka sekarang?"

Sekarang pertanyaannya berubah menjadi:

"Setelah trade dibuka, apakah trade tersebut masih layak dipertahankan, dan jika iya, bagaimana SL, TP, serta ukuran posisi harus dikelola?"

Ini bukan lagi masalah *entry optimization*. Kita sekarang membangun **dynamic position management**.

Arsitektur kita menjadi:

Entry Optimization → Execution → Position Management → Exit

dan Position Management sendiri akan terus berjalan sepanjang umur posisi.

O.1. Mengapa position management harus dipisahkan dari entry?

Misalkan pada waktu:

t_0

model menghasilkan:

$$x_{t_0}^* = \text{LONG BTC.}$$

Order kemudian filled.

Pada:

$$t_1 = t_0 + 1 \text{ jam,}$$

kondisi pasar berubah.

Misalnya:

$$\hat{\mu}_{BTC,t_1} < \hat{\mu}_{BTC,t_0}.$$

Tetapi BTC masih berada:

$$+1.8R$$

dari entry.

Apakah kita langsung menjual?

Tidak.

Karena informasi yang relevan sekarang bukan hanya:

$$\hat{\mu}_{t_1},$$

tetapi juga:

unrealized PnL,

distance to SL,

distance to TP,

time in trade,

updated volatility,

updated factor regime.

Jadi kita membutuhkan state tambahan.

O.2. State posisi

Untuk setiap posisi i , kita definisikan:

$$s_i(t) = [P_i^{entry}, P_i(t), SL_i(t), TP_i(t), q_i(t), \tau_i(t), PnL_i(t), \sigma_i(t), \mu_i(t), R_i(t)]$$

dengan:

- P_i^{entry} : harga entry,
- $P_i(t)$: harga sekarang,
- $SL_i(t)$: stop-loss sekarang,
- $TP_i(t)$: take-profit sekarang,
- $q_i(t)$: quantity,
- τ_i : umur posisi,
- PnL_i : unrealized PnL,
- σ_i : volatility terkini,
- μ_i : expected return terkini,
- R_i : remaining risk.

Maka sistem tidak lagi hanya melihat:

market state.

Ia melihat:

market state + position state.

O.3. Normalisasi PnL menggunakan R

Karena sejak Tahap N kita menggunakan risk budget, ukuran yang sangat berguna adalah:

$$R_i(t) = \frac{PnL_i(t)}{R_i^{initial}}$$

Misalnya initial risk:

$$R_i^{initial} = \$1.$$

Jika floating loss:

$$PnL = -\$0.5,$$

maka:

$$R_i = -0.5R.$$

Jika profit:

$$PnL = +\$2,$$

maka:

$$R_i = +2R.$$

Ini jauh lebih konsisten daripada menggunakan dollar PnL secara langsung.

O.4. Mengapa R -multiple sangat penting?

Misalkan:

Trade A:

$$+\$10$$

dengan initial risk:

$$\$20.$$

Maka:

$$R_A = 0.5R.$$

Trade B:

$$+\$10$$

dengan initial risk:

\$2.

Maka:

$$R_B = 5R.$$

Secara nominal sama:

\$10,

tetapi secara risk-adjusted sangat berbeda.

Karena sistem kita memang dibangun berdasarkan:

\$1 modeled risk

maka seluruh position management sebaiknya menggunakan R .

O.5. Tiga keadaan posisi

Kita dapat membagi posisi menjadi:

State I - Floating Loss

$$R_i < 0$$

State II - Floating Profit

$$R_i > 0$$

State III - Near Exit

Misalnya:

$$P_i \rightarrow TP_i$$

atau:

$$P_i \rightarrow SL_i.$$

Tetapi kita tidak boleh menggunakan PnL saja.

Kita juga perlu mengetahui apakah **thesis** trade masih valid.

O.6. Thesis score

Definisikan:

$$\Theta_i(t)$$

sebagai ukuran seberapa kuat alasan awal trade masih berlaku.

Misalnya:

$$\Theta_i = f(\mu_i, factor_i, momentum_i, volatility_i, liquidity_i, correlation_i).$$

Secara sederhana:

$$\Theta_i > 0$$

berarti thesis masih mendukung posisi.

Sedangkan:

$$\Theta_i < 0$$

berarti thesis mulai invalid.

O.7. Kita tidak ingin threshold arbitrer

Daripada:

Kalau profit 2%, geser SL.

lebih matematis jika menggunakan:

$$R,$$

volatility,

dan expected future return.

Jadi keputusan tidak bergantung pada:

2%.

Melainkan pada keadaan relatif terhadap:

$$\sigma_t$$

dan:

$$R_i.$$

O.8. Stop-loss awal

Saat entry:

$$P_0.$$

Untuk long:

$$SL_0 = P_0 - d.$$

Risk:

$$R_0 = qd.$$

Kita telah menetapkan:

$$R_0 \leq \$1.$$

Stop ini disebut:

$$\boxed{SL_{initial}}.$$

O.9. Jangan langsung mempersempit SL ketika floating loss

Ini prinsip yang penting.

Misalnya:

$$R = -0.5R.$$



Kemudian model memperbarui prediksi.

Jika thesis masih valid:

$$\Theta > 0,$$

maka kita tidak boleh otomatis melakukan:

$$SL \rightarrow P_{entry}.$$

Karena itu dapat mengubah trade yang secara statistik masih valid menjadi stop-out hanya karena noise.

O.10. Floating loss decision

Kita definisikan future expected utility:

$$U_i(t) = E[\Pi_i^{future} | F_t].$$

Jika:

$$U_i(t) > 0$$

dan:

$$\Theta_i(t) > 0,$$

maka posisi masih memiliki alasan untuk dipertahankan.

Sebaliknya jika:

$$U_i(t) < 0,$$

thesis tidak lagi mendukung posisi.

O.11. Keputusan exit yang lebih matematis

Kita dapat mendefinisikan:

$$V_i^{hold}(t) = E[\Pi_i^{future} | F_t] - C_i^{future}.$$

Jika:

$$V_i^{hold} > 0,$$

hold memiliki expected value positif.

Jika:

$$V_i^{hold} < 0,$$

exit menjadi lebih rasional.

O.12. Opportunity cost

Tetapi ada satu hal lagi.

Modal yang digunakan oleh posisi lama tidak dapat digunakan untuk trade baru.

Misalkan ada candidate trade baru j dengan:

$$V_j^{entry}.$$

Maka kita dapat membandingkan:

$$V_i^{hold}$$

dengan:

$$V_j^{entry}.$$

Definisikan:

$$OC_{i \rightarrow j} = V_j^{entry} - V_i^{hold}.$$

Jika:

$$OC > 0,$$

trade baru memiliki expected value lebih tinggi.

Ini menghasilkan konsep:

dynamic capital reallocation.

O.13. Jadi floating loss tidak otomatis berarti exit

Kita memperoleh aturan konseptual:

Loss $\Rightarrow$ Exit.

Yang menentukan adalah:

Future Expected Value

dan:

Thesis Validity.

O.14. Sekarang floating profit

Misalkan:

$$R_i = +1.5R.$$

Pertanyaan kita:

Apakah kita mengambil profit sekarang?

atau:

Apakah kita membiarkan profit berkembang?

Secara matematis kita membandingkan:

V_{hold}

dengan:

V_{exit} .



O.15. Immediate exit value

Jika kita keluar sekarang:

$$V_{exit}(t) = P n L_i(t) - C_{exit}.$$

Sedangkan jika hold:

$$V_{hold}(t) = E[P n L_i(T) | F_t] - C_{future}.$$

Maka:

$$\Delta V = V_{hold} - V_{exit}.$$

O.16. Jika $\Delta V > 0$

Lebih baik mempertahankan posisi.

Jika:

$$\Delta V < 0,$$

lebih baik exit.

Ini memberikan dasar matematis untuk:

"floating profit tinggi, apakah TP harus dinaikkan?"

O.17. TP bukan angka statis

Awalnya:

$$TP_0.$$

Tetapi setelah market berubah:

$$TP_t$$

dapat menjadi:

$$TP_t > TP_0.$$

Namun kenaikan TP hanya dilakukan jika:

$$E[P_{future}]$$

masih mendukung target tersebut.

O.18. Kita membutuhkan target distribution

Daripada hanya memprediksi:

$$\hat{\mu}_t,$$

lebih ideal kita memiliki:

$$p(\Delta P_{t \rightarrow T} \mid F_t).$$

Dari distribusi tersebut kita dapat menghitung:

$$P(P_T > TP).$$

Dan:

$$P(P_T < SL).$$

Ini jauh lebih berguna untuk position management.

O.19. Expected value dengan TP dan SL

Misalkan:

$$p_T = P(TP \text{ hit}),$$

$$p_S = P(SL \text{ hit}),$$

dan:

$$p_H = 1 - p_T - p_S$$

untuk kondisi lainnya.

Expected PnL:

$$EV = p_T G - p_S L + p_H E_H.$$

Jika:

$$EV > 0,$$

target tersebut masih layak.

O.20. Dynamic TP

Misalkan kandidat TP:

$$TP^{(1)}, TP^{(2)}, TP^{(3)}.$$

Kita dapat menghitung:

$$EV^{(1)}, EV^{(2)}, EV^{(3)}.$$

Kemudian:

$$TP^* = \arg \max_{TP^{(k)}} EV^{(k)}.$$

Jadi TP tidak lagi:

"naik 2% setiap kali profit."

Tetapi:

pilih target yang memiliki expected utility tertinggi berdasarkan state pasar sekarang.

O.21. Dynamic Stop-Loss

Sekarang masalah yang lebih menarik.

Kita ingin memilih:

$$SL_t.$$

Kandidatnya dapat berupa:

$$SL^{(1)} = SL_{initial},$$

$$SL^{(2)} = BEP,$$

$$SL^{(3)} = \textit{trailing stop},$$

$$SL^{(4)} = \textit{volatility stop}.$$

O.22. Break-even stop

Untuk long:

$$SL_t = P_{entry}.$$

Namun sebenarnya BEP harus memperhitungkan biaya.

Jika:

$$C_{entry} + C_{exit}$$

tidak nol, maka break-even sebenarnya:

$$P_{BEP} = P_{entry} + \frac{C_{total}}{q}.$$

Jadi:

$$SL = P_{entry}$$

belum tentu benar-benar break-even.

O.23. Trailing stop berbasis volatility

Misalkan:

$$ATR_t$$

atau volatility scale:

$$\sigma_t.$$

Untuk long:

$$SL_t = P_t - k\sigma_t P_t$$

dengan:

$$k > 0.$$

Tetapi SL hanya boleh bergerak satu arah:

$$SL_t \geq SL_{t-1}$$

untuk long.

Untuk short:

$$SL_t \leq SL_{t-1}.$$

O.24. Mengapa tidak hanya memakai trailing stop?

Karena trailing stop tidak mengetahui thesis.

Misalnya volatility meningkat.

Trailing stop dapat menjadi:

$$SL_t = P_t - k\sigma_t P_t.$$

Jika σ_t meningkat tajam, stop justru dapat bergerak **lebih jauh**.

Maka position management harus mempertimbangkan:

$$Price + Volatility + Thesis + Expected Value.$$

O.25. Kandidat action

Sekarang kita formalasikan decision space.

Untuk posisi yang sedang terbuka, kita memiliki:

$$a_i \in A.$$

Misalnya:

$$A = \{KEEP, EXIT, BEP, TRAIL, TP_UP, PARTIAL_EXIT\}.$$

O.26. Binary encoding

Definisikan:

$$z_{i,a} \in \{0, 1\}.$$

Dengan constraint:

$$\sum_{a \in A} z_{i,a} = 1.$$

Artinya satu action dipilih.

O.27. Utility setiap action

Untuk action a :

$$U_{i,a} = E[\Pi_i \mid a, F_t] - C_{i,a} - Risk_{i,a}.$$

Kemudian kita ingin:

$$a_i^* = \arg \max_a U_{i,a}.$$

O.28. Ini dapat menjadi QUBO kedua

Ubah menjadi minimization:

$$E_i(z) = - \sum_a U_{i,a} z_{i,a} + P \left(\sum_a z_{i,a} - 1 \right)^2.$$

Dengan:

$$z_{i,a} \in \{0, 1\}.$$

Ini adalah QUBO.

O.29. Tetapi kita dapat membuatnya jauh lebih kuat

Jika kita memiliki beberapa posisi:

$$i = 1, \dots, N_p,$$

maka setiap action memiliki interaksi.

Misalnya:

exit BTC memungkinkan capital dialihkan ke ETH.

Maka:

$$U_{i,a}$$

tidak lagi independen.

Kita membutuhkan:

$$U(z) = \sum_{i,a} u_{i,a} z_{i,a} + \sum_{i,j,a,b} J_{i,a,j,b} z_{i,a} z_{j,b}.$$

Dan kembali kita mendapatkan QUBO.

O.30. Position management QUBO

Secara umum:

$$E_{\text{manage}}(z) = -z^T U z + P(z)$$

atau sesuai konvensi:

$$E_{\text{manage}}(z) = - \sum_{i,a} U_{i,a} z_{i,a} + \sum_{i,j,a,b} C_{i,a,j,b} z_{i,a} z_{j,b} + P(z).$$

O.31. Apa yang dimaksud interaction antar-action?

Misalnya dua posisi:

BTC

dan:

ETH.

Jika BTC ditutup:

EXIT_{BTC},

capital tersedia.

Jika ETH dipertahankan:

KEEP_{ETH},

tidak ada masalah.

Tetapi jika:

KEEP_{BTC}

dan:

ENTER_{SOL},

total risk mungkin melebihi:

$$R_{portfolio}.$$

Maka interaction penalty muncul.

O.32. Dynamic portfolio risk

Pada waktu t :

$$w_t$$

adalah posisi aktual.

Jika kita mempertimbangkan action a , kita memperoleh:

$$w_t^{(a)}.$$

Risk:

$$\sigma_p^2(a) = w_t^{(a)T} \Sigma_t w_t^{(a)}.$$

Maka action yang meningkatkan risk terlalu besar dapat dihukum.

O.33. Remaining risk

Ini sangat penting untuk sistem kita.

Jika:

$$P_t > P_{entry}$$

untuk long, dan SL telah dinaikkan:

$$SL_t > P_{entry},$$

maka posisi sudah memiliki:

$$locked\ profit.$$

Define:

$$L_i(t) = q_i(SL_i(t) - P_{entry}).$$

Untuk long.

Jika:

$$L_i > 0,$$

minimum realized profit setelah SL tersentuh sudah positif, mengabaikan gap/slippage.

O.34. Locked profit

Dalam R -multiple:

$$R_{locked} = \frac{L_i}{R_{initial}}.$$

Misalnya:

$$R_{locked} = 0.8R.$$

Artinya meskipun trade berbalik dan menyentuh SL, secara model kita masih mempertahankan:

$$+0.8R.$$

O.35. Ini mengubah objective

Sebelum profit:

$$R_{locked} \leq 0.$$

Setelah BEP:

$$R_{locked} \approx 0.$$

Setelah trailing:

$$R_{locked} > 0.$$



Maka position management harus memaksimalkan:

$$\boxed{\text{Expected additional profit}}$$

tanpa mengorbankan terlalu banyak:

$$\boxed{\text{locked profit.}}$$

O.36. Objective baru

Definisikan:

$$G_i(a) = E[\text{future PnL} \mid a].$$

dan:

$$L_i(a) = \text{locked profit under } a.$$

Kita dapat membuat:

$$\boxed{U_i(a) = G_i(a) + \eta L_i(a) - \lambda \text{Risk}_i(a).}$$

Parameter:

$$\eta$$

mengontrol seberapa penting capital protection.

O.37. Tetapi kita harus menghindari "profit fixation"

Jika:

$$\eta$$

terlalu besar, sistem akan terlalu cepat:

EXIT.

Jika:

$$\eta$$

terlalu kecil, sistem akan membiarkan profit menguap.

Jadi:

$$\eta$$

adalah parameter yang harus diuji melalui walk-forward validation.

O.38. Dynamic TP dan SL sebagai joint optimization

Jangan memilih:

$$TP$$

dan:

$$SL$$

secara terpisah.

Karena keduanya saling berhubungan.

Kita definisikan action:

$$a = (SL_k, TP_l).$$

Kemudian:

$$a \in A.$$

Misalnya:

$$A = \{(SL_1, TP_1), (SL_1, TP_2), (SL_2, TP_2), (SL_3, TP_3), \dots\}.$$

O.39. Expected utility

Untuk setiap kombinasi:

$$(SL_k, TP_l),$$

hitung:

$$U_{kl} = EV(SL_k, TP_l) - Cost_{kl} - Risk_{kl}.$$

Kemudian:

$$(SL^*, TP^*) = \arg \max_{k,l} U_{kl}.$$

O.40. Ini menjawab pertanyaan awalmu

Kamu sebelumnya membayangkan:

jika floating profit tinggi, mungkin TP dinaikkan lagi.

Jawabannya sekarang menjadi:

Ya, tetapi bukan karena profit sudah tinggi.

TP dinaikkan jika:

$$U(TP_{new}, SL_{new}) > U(TP_{old}, SL_{old}).$$

Jadi keputusan tersebut berasal dari **expected future utility**, bukan dari rule arbitrer.

O.41. Time stop

Ada kondisi lain:

Trade tidak loss, tetapi juga tidak bergerak.

Misalnya:

$$R \approx 0$$

selama:

$$\tau = 8 \text{ jam.}$$

Capital menjadi terkunci.

Kita definisikan:

$$U_{time} = E[\Pi_{future}] - \lambda_{\tau} \tau.$$

Jika:

$$U_{time} < 0,$$

exit dapat menjadi optimal.

O.42. Opportunity cost kembali masuk

Semakin lama posisi stagnan:

$$\tau \uparrow,$$

semakin besar opportunity cost.

Jadi:

$$Time \text{ in trade}$$

adalah variabel matematis, bukan sekadar informasi tambahan.

O.43. Regime change

Ini bahkan lebih penting.

Misalkan ketika entry:

$$Regime_t = Bull.$$

Kemudian:

$$Regime_{t+1} = Neutral/Bear.$$

Expected return:

$$\hat{\mu}_t$$

dapat berubah drastis.

Maka:

$$U_{hold}$$

harus dihitung ulang.

O.44. Regime-conditioned management

Kita dapat menulis:

$$U_i(a \mid z_t)$$

dengan:

$$z_t$$

adalah market regime.

Contoh:

$$z_t \in \{Trend, MeanReversion, HighVol, LowVol, RiskOn, RiskOff\}.$$

O.45. Gold dan DXY juga kembali masuk

Karena sebelumnya kita ingin sistem bekerja tidak hanya pada crypto tetapi juga saham internasional, factor layer kita dapat memberi:

$$F_t = [BTC, ETH, Gold, DXY, SPX, VIX, \dots].$$

Jika misalnya:

$$DXY \uparrow$$

dan:

Gold ↓,

model dapat memperbarui:

$$\hat{\mu}_i,$$

$$\Sigma_i,$$

dan akhirnya:

$$U_i(a).$$

Dengan demikian perubahan faktor global dapat memicu:

position management.

O.46. Dynamic rebalancing

Sekarang kita dapat mendefinisikan proses periodik.

Pada waktu:

$$t.$$

Kita memiliki:

$$P_t = \text{all open positions.}$$

Update market:

$$F_t.$$

Kemudian:

$$\hat{\mu}_t, \Sigma_t$$

dihitung ulang.

O.47. Langkah pertama: valuasi posisi

Untuk setiap posisi:

$i :$

hitung:

$PnL_i,$

$R_i,$

$R_{locked,i},$

$\tau_i,$

$\Theta_i,$

$U_{hold,i}.$

O.48. Langkah kedua: generate candidate actions

Untuk setiap posisi:

$$A_i = \{KEEP, EXIT, BEP, TRAIL, TP_1, TP_2, PARTIAL\}.$$

O.49. Langkah ketiga: bangun management QUBO

$$Q_t^{manage} = Q(PnL_t, \hat{\mu}_t, \Sigma_t, SL_t, TP_t, \tau_t, Liquidity_t, Factor_t).$$

O.50. Langkah keempat: solve

Kita dapat menggunakan:

$$x_t^{manage} = \arg \min_x x^T Q_t^{manage} x.$$

Solver dapat berupa:

- exhaustive search untuk problem kecil,
- classical QUBO solver,
- simulated annealing,
- tabu search,
- MILP,
- QAOA.

Dengan demikian quantum bukan satu-satunya solver.

O.51. Langkah kelima: execution

Jika hasilnya:

KEEP :

tidak melakukan apa-apa.

Jika:

BEP :

modify SL.

Jika:

TP_UP :

modify TP.

Jika:

PARTIAL :

reduce quantity.

Jika:

EXIT :

close position.

O.52. Setelah execution

Kita kembali ke:

Market Monitoring.

Sehingga terdapat loop:

$Observe \rightarrow Estimate \rightarrow Optimize \rightarrow Execute \rightarrow Monitor \rightarrow Re-optimize.$
--

Ini adalah inti sistem kita.

O.53. Full mathematical loop

Secara matematis:

$$F_t \rightarrow \theta_t \rightarrow (\mu_t, \Sigma_t)$$

kemudian:

$$(\mu_t, \Sigma_t) \rightarrow Q_t^{entry}$$

kemudian:

$$Q_t^{entry} \rightarrow x_t^*$$

kemudian execution menghasilkan:

$$s_t.$$

Kemudian:

$$(s_t, F_t) \rightarrow Q_t^{manage}$$

dan:

$$Q_t^{manage} \rightarrow a_t^*.$$

O.54. Kita sekarang memiliki dua Hamiltonian

Entry:

$$H_C^{entry}(t)$$

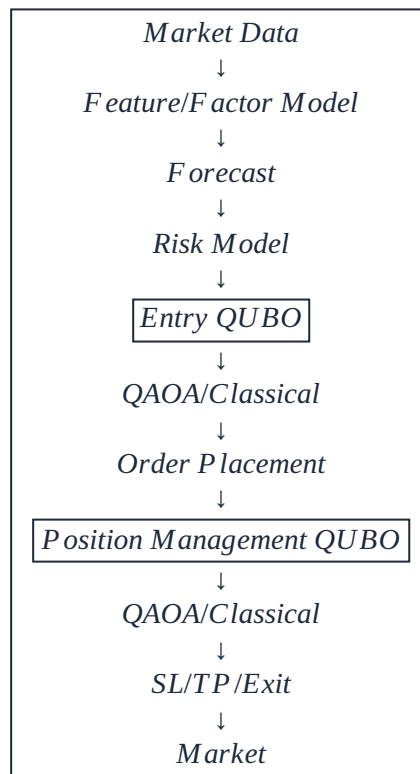
Management:

$$H_C^{manage}(t)$$

Keduanya berasal dari market state yang sama tetapi memiliki tujuan berbeda.

O.55. Ini memungkinkan arsitektur yang jauh lebih menarik

Secara keseluruhan:



O.56. Namun ada satu hal yang harus kita cegah: excessive re-optimization

Jika kita menjalankan optimizer setiap:

1 detik,

sistem mungkin:

$KEEP \rightarrow EXIT \rightarrow KEEP \rightarrow EXIT$

hanya karena noise.

Ini disebut:

$decision\ churning.$

O.57. Hysteresis

Kita dapat membuat threshold.

Misalnya keputusan baru hanya dilakukan jika:

$$U_{new} - U_{current} > \delta.$$

Dengan:

$$\delta > 0.$$

Jadi optimizer tidak mengubah posisi hanya karena improvement sangat kecil.

O.58. Transaction cost secara alami menghasilkan hysteresis

Sebenarnya kita dapat membuat:

$$U_{switch} = U_{new} - U_{current} - C_{switch}.$$

Switch hanya jika:

$$U_{switch} > 0.$$

Jadi biaya transaksi sendiri membantu mencegah overtrading.

O.59. Rebalancing bukan berarti selalu mengubah posisi

Ini penting.

Dynamic rebalancing berarti:

re-evaluate

bukan:

always trade.

Jika posisi saat ini masih optimal:

$$a^* = KEEP.$$

Maka:

No action.

O.60. Position management sebagai optimal stopping

Ada perspektif matematika yang lebih dalam.

Pertanyaan:

Kapan sebaiknya saya keluar?

sebenarnya adalah masalah:

Optimal Stopping.

Kita memiliki state:

$$X_t = (P_t, \mu_t, \sigma_t, \Theta_t, \tau_t, \dots).$$



Kemudian kita mencari stopping time:

$$\tau^* = \inf \{t : V(X_t) = G(X_t)\}.$$

dengan:

- $V(X_t)$: value of continuing,
- $G(X_t)$: value of exiting.

O.61. Ini sangat penting secara teoritis

Jadi position management kita sebenarnya dapat dipandang sebagai:

$$\text{Dynamic Programming} + \text{Optimal Stopping} + \text{Combinatorial Optimization}.$$

Dan QUBO adalah representasi diskrit dari keputusan kombinatorial tersebut.

O.62. Bahkan kita bisa melihatnya sebagai MDP

State:

$$s_t.$$

Action:

$$a_t.$$

Reward:

$$r_t.$$

Transition:

$$P(s_{t+1}|s_t, a_t).$$

Maka kita memiliki:

$$\text{Markov Decision Process}.$$

Namun kita **tidak harus menggunakan reinforcement learning**.

Kita dapat menggunakan probabilistic forecast + optimization.

Ini lebih mudah dijelaskan dan divalidasi untuk penelitian kuantitatif.

O.63. Hubungan dengan QAOA

QAOA tidak menggantikan keseluruhan MDP.

QAOA menyelesaikan subproblem:

Which action configuration is optimal at t ?

Jadi:

$MDP \rightarrow Optimization\ Subproblem \rightarrow QUBO \rightarrow QAOA.$

Ini framing yang jauh lebih tepat.

O.64. Contoh konkret

Misalkan:

$$R_{initial} = \$1.$$

Posisi BTC:

$$R = +2.4R.$$

Thesis score:

$$\Theta = 0.72.$$

Expected future return:

$$\mu > 0.$$

Current SL:

$$+0.8R$$

locked.

Candidate actions:

Action	Locked profit	Expected future profit	Risk
EXIT	$2.4R$	$\$0\$$	0
KEEP	$0.8R$	$1.1R$	tinggi
TRAIL	$1.2R$	$0.8R$	sedang
TP-UP	$0.8R$	$1.5R$	tinggi

Model menghitung:

$$U(a).$$

Misalnya hasil:

$$U_{EXIT} = 2.35,$$

$$U_{KEEP} = 1.90,$$

$$U_{TRAIL} = 2.00,$$

$$U_{TP-UP} = 2.15.$$

Maka:

EXIT

menjadi optimal.

Bukan karena:

profit sudah $2.4R$.

Tetapi karena:

expected value of continuing < value of exiting.

O.65. Contoh kedua

Misalkan:

$$R = +2.4R$$

tetapi:

$$\Theta = 0.91,$$

dan expected future return masih tinggi.

Mungkin:

$$U_{KEEP} = 2.8$$

dan:

$$U_{EXIT} = 2.35.$$

Maka:

$$\boxed{KEEP.}$$

Bahkan:

$$TP$$

dapat dinaikkan jika:

$$U(TP_{new}) > U(TP_{old}).$$

O.66. Contoh floating loss

Misalkan:

$$R = -0.6R.$$

Tetapi:

$$\Theta = 0.85,$$

dan:

$$U_{hold} = 0.3.$$

Sedangkan:

$$U_{exit} = -0.6.$$

Maka:

KEEP.

Ini menjawab bagian awal desainmu:

floating loss tidak otomatis ditutup.

O.67. Tetapi jika thesis rusak

Misalnya:

$$R = -0.6R,$$

namun:

$$\Theta = -0.8,$$

dan:

$$U_{hold} = -1.2.$$

Sedangkan:

$$U_{exit} = -0.6.$$

Maka:

EXIT.

Jadi stop-loss bukan satu-satunya exit mechanism.

O.68. Kita sekarang bisa mendefinisikan empat alasan exit

1. Risk exit

$$P_t \leq SL_t.$$

2. Target exit

$$P_t \geq TP_t.$$

3. Thesis exit

$$\Theta_t < \Theta_{min}.$$

4. Opportunity-cost exit

$$V_{new} > V_{hold} + C_{switch}.$$

Ini jauh lebih kuat daripada hanya:

$$TP/SL.$$

O.69. Tetapi SL/TP tetap menjadi safety layer

Ini penting untuk deployment.

Optimizer tidak boleh menjadi satu-satunya proteksi.

Jika:

$$QAOA$$

hang,

atau:

$$network$$

putus,

atau:

model

menghasilkan output aneh,

SL yang sudah terpasang tetap harus bekerja.

Jadi:

Hard Risk Controls

harus berada di bawah:

Optimization Layer.

O.70. Hierarki kontrol

Saya menyarankan arsitektur:

Hard Risk Layer

di paling bawah.

Di atasnya:

Execution Layer

kemudian:

Position Management

kemudian:

Entry Optimization

dan paling atas:

Forecast/Research

Sehingga optimizer tidak pernah bisa berkata:

"Saya yakin sekali, jadi abaikan maximum loss."

Constraint risiko tetap menang.

O.71. Tahap O dalam bentuk algoritma

Secara konseptual:

```
INITIALIZE position state
```

```
WHILE market is open:
```

```
    observe market
```

```
    update:
```

```
        price
```

```
        volatility
```

```
        factors
```

```
        expected return
```

```
        covariance
```

```
        liquidity
```

```
    FOR every open position:
```

```
        calculate:
```

```
            PnL
```

```
            R-multiple
```

```
            locked profit
```

```
            time-in-trade
```

```
            thesis score
```

```
            future expected value
```

```
        generate candidate actions:
```

```
            KEEP
```

```
            EXIT
```

```
            BEP
```

```
            TRAIL
```

```
            TP-UP
```

```
            PARTIAL EXIT
```

```
    construct management QUBO
```

```
    solve QUBO
```

```
    IF new decision is sufficiently better:
```

```
        execute action
```

```
    update SL/TP/state
```

```
periodically:
```

rebuild entry QUBO
evaluate new opportunities

END

O.72. Arsitektur final setelah Tahap O

Sekarang sistem kita telah berkembang menjadi:

Market Data → *Feature Engineering*
→ *Factor Model*
→ *Forecast*
→ *Risk Model*
→ *Entry QUBO*
→ *QAOA/Classical*
→ *Order Placement*
→ *Position State*
→ *Management QUBO*
→ *QAOA/Classical*
→ *SL/TP/Exit*
→ *Rebalancing.*

O.73. Dan sekarang ada sesuatu yang sangat menarik

Sebelum Tahap O, kita hanya mempunyai:

Static Portfolio Optimization.

Setelah Tahap O, kita mulai memiliki:

Dynamic Sequential Portfolio Optimization.

Artinya keputusan pada waktu t mempengaruhi state pada:

$t + 1$.

Misalnya:

$$EXIT_{BTC}$$

membebaskan capital.

Capital tersebut dapat digunakan untuk:

$$BUY_{ETH}.$$

Jadi keputusan kita memiliki **intertemporal effect**.

O.74. Persamaan konseptual terpenting Tahap O

Saya akan merangkum Tahap O dengan satu persamaan:

$$a_t^* = \arg \max_{a \in A} [E(\Pi_{t:T} \mid s_t, a) - C(a) - \lambda R(a) + \eta L(a) - \lambda_t \tau].$$

dengan:

$$s_t = \{P_t, PnL_t, SL_t, TP_t, \hat{\mu}_t, \Sigma_t, \Theta_t, \tau_t, Liquidity_t, Factor_t\}.$$

Kemudian keputusan diskrit tersebut kita transformasikan menjadi:

$$Q_t^{manage}$$

dan akhirnya:

$$Q_t^{manage} \rightarrow H_C^{manage} \rightarrow QAOA.$$

O.75. Posisi kita sekarang

Kita sudah memiliki dua problem optimasi yang berbeda:

$$\text{Problem 1 - Entry}$$

$$\hat{\mu}_t, \Sigma_t \rightarrow Q_t^{entry} \rightarrow \text{trade selection.}$$

dan:

Problem 2 - Management

$$s_t, \hat{\mu}_t, \Sigma_t \rightarrow Q_t^{manage} \rightarrow \text{KEEP/EXIT/SL/TP/PARTIAL.}$$

Keduanya dapat diuji dengan:

Exact vs Classical vs QAOA

pada problem yang **identik**.

Dan menurut saya, ini adalah titik di mana proyek kita mulai menjadi jauh lebih serius.

Kita tidak lagi sekadar membuat:

"bot trading yang memakai QAOA."

Kita sedang membangun sebuah **dynamic quantitative decision system**, di mana:

market dynamics → statistical estimation → risk → combinatorial optimization → quantum optimization → exec

dan setiap keputusan dapat ditulis secara matematis.

Yang juga sangat penting: **QAOA hanyalah solver**, bukan sumber alpha. Alpha berasal dari data, factor model, forecasting, dan risk model. QAOA menjawab pertanyaan kombinatorial: *dari seluruh keputusan diskrit yang feasible, mana yang paling baik menurut objective kita?*

Dengan Tahap O selesai, fondasi matematis untuk **entry + position management** sudah terbentuk. Tahap berikutnya secara alami adalah **Tahap P - Execution & Microstructure Optimization**, yaitu membawa keputusan abstrak seperti "*LONG BTC dengan risk \$1*" menjadi keputusan nyata: **harga limit order, kedalaman order book, probabilitas fill, expected slippage, queue position, cancel/replace, dan kapan sebuah order sebaiknya tidak diletakkan sama sekali**. Di situlah sistem kita mulai benar-benar bersentuhan dengan *short-horizon trading* dan market microstructure.